

Detecção de Anomalias em Licitações Públicas

Priorizar, com IA, quais licitações merecem o olhar do auditor

EQUIPE

César Augusto de Carvalho Calafrioli Móes

Anthony Silva Ferreira

Gustavo Camara Bastos

Jhonata Lima de Souza

Paulo Brandão

George Queiroz

Pedro Roncoli Sarment Moreira

Bruno Oliveira

Problema: milhares de licitações por mês, equipe de auditoria limitada — hoje a escolha do que auditar vem de denúncia ou amostragem, sem critério de risco.

Papel da IA: não existe rótulo de fraude → aprendizado não supervisionado. O modelo aprende o padrão normal e ranqueia os desvios.

Dados e solução proposta

Portal da Transparência (CGU) — 4 arquivos ZIP mensais, jan–abr/2024

8.064

licitações

unidade de análise

175.689

itens

agregados por licitação

508.824

participantes

concorrência por licitação

Dataset final e saída

Matriz 8.064×18 features — 1 linha = 1 licitação.

Entrada: 3 tabelas CSV.

Saída: ranking por score de anomalia (0–1) + bandeiras vermelhas.

Variável prevista: nenhuma

Não há rótulo de fraude na base pública. Por isso o problema é de detecção de anomalias não supervisionada, a saída é um score contínuo, não uma classe.

Duas ressalvas declaradas desde já

- A tabela EmpenhosRelacionados foi descartada: vem com 0 linhas nos arquivos utilizados.
- O pipeline de src/ produz 18 features, mas os experimentos dos notebooks (entregas 3 e 4) usaram um dataset intermediário de 1 feature (dias entre abertura e resultado). Retomamos isso no slide 11.

3

Requisitos funcionais

Elicitados na tarefa de Engenharia de Requisitos com GR4ML (Fase 4)

RF-01

O sistema deve calcular um **score de anomalia para cada edital de licitação** publicado, a partir do valor estimado tratado, do tempo decorrido calculado e das modalidades de compra vetorizadas (*todas na escala normalizada de 0 a 1*), para permitir que o **Auditor decida se prioriza para aprofundamento, coloca em fila ou descarta uma investigação**

IMPLEMENTADO

RF-03

O sistema deve **classificar a significância do desvio do edital** (*categorizando-o como alta prioridade, média prioridade ou descarte*), a partir da comparação do score gerado contra limiares (thresholds) de contaminação pré-definidos e ajustáveis pelo usuário, para permitir que o **Auditor otimize o esforço de triagem concentrando-se apenas nos casos de maior relevância material**.

IMPLEMENTADO

RF-02

Exibir atribuição SHAP por atributo do edital sinalizado

ROADMAP

RF-04

Calcular score de risco por fornecedor concorrente

ROADMAP

RF-05

Cruzar CNPJ com sanções ativas nas bases CEIS / CNEP

ROADMAP

RF-06

Comparar desvios com tipologias de fraude conhecidas

ROADMAP

O professor definiu o projeto como uma maquete funcional: implementamos o núcleo (score e priorização) e declaramos o restante como roadmap.

4

Requisitos não funcionais e critérios de aceitação

Cada RNF tem um critério verificável — não é intenção, é teste

1

Reprodutibilidade

Mesma semente produz o mesmo resultado

como verificamos: teste test_treino_e_reprodutivel

PASSA

2

Testabilidade

Suíte cobrindo dados, pré-processamento, modelo e treino

como verificamos: python -m unittest discover -s tests

26 TESTES, OK

3

Modularidade

Cada pacote com uma responsabilidade única

como verificamos: trocamos o modelo sem tocar no pipeline

8 PACOTES

4

Desempenho

Processar o lote mensal com tempestividade

como verificamos: medição do pipeline completo

0,27 s

5

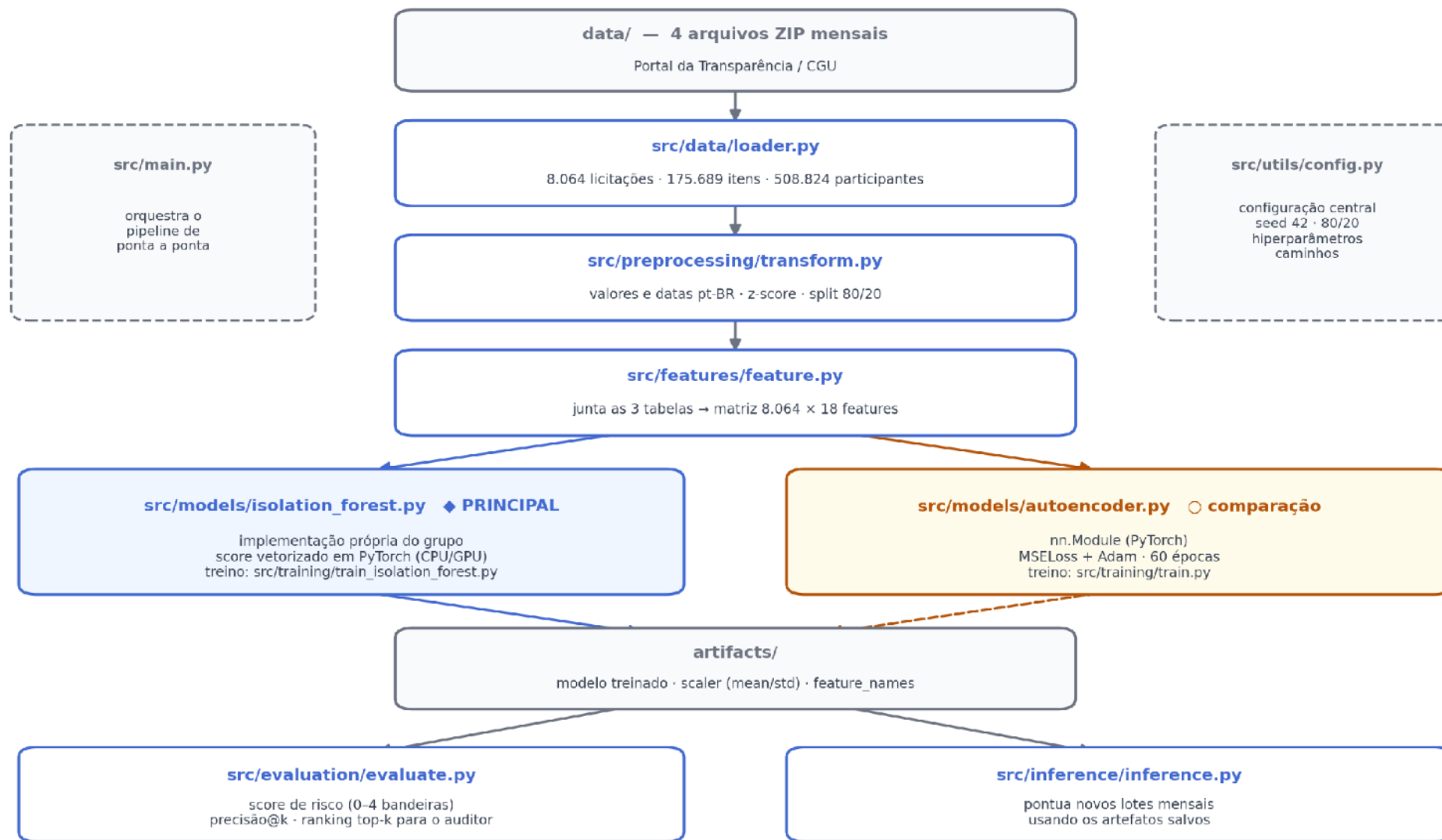
Explicabilidade

O auditor precisa saber por que foi sinalizado

como verificamos: régua de risco 0–4 em cada alerta

PARCIAL

Arquitetura do sistema



Cada pasta de src/ tem uma responsabilidade única; a comunicação entre camadas é por arrays NumPy (baixo acoplamento).

Decisões arquiteturais

Fluxo dos dados

ZIP → tabelas NumPy → matriz 8.064×18 → score → ranking

Onde pré-processa

preprocessing/transform.py e features/feature.py

Onde treina

training/ — um script por modelo

Onde infere

inference/inference.py, com os artefatos salvos (sem re-treinar)

Onde o modelo é salvo

artifacts/ — modelo + scaler.npz (mean/std) + feature_names.json

Como as camadas conversam

arrays NumPy — nenhum objeto acoplado entre camadas

Baixo acoplamento, comprovado no próprio projeto

Trocamos o modelo principal de Autoencoder para Isolation Forest mexendo apenas em models/ e training/. O loader, o pré-processamento, as features e a avaliação não mudaram uma única linha, é separação de responsabilidades funcionando na prática.

7

Estrutura do projeto

```
eng-soft-ia-fw-2026/
├── data/           4 ZIPs mensais (CGU)
├── docs/           dicionário + GR4ML
├── notebook/       exploração e experimentos
├── src/
│   ├── data/       loader · dataset
│   │               prepare_data
│   ├── preprocessing/ transform
│   ├── features/    feature
│   ├── models/      isolation_forest
│   │               autoencoder
│   ├── training/    train
│   │               train_isolation_forest
│   ├── evaluation/  evaluate
│   ├── inference/   inference
│   ├── utils/       config
│   └── main.py
├── tests/          4 suítes · 26 testes
├── requirements.txt
└── README.md
```

Por que separado assim

Cada pasta isola um motivo de mudança.

Trocar a fonte de dados mexe só em data/.

Trocar o modelo mexe só em models/ e training/, foi exatamente o que fizemos.

Não é só organização

A estrutura é o que tornou possível comparar dois modelos completamente diferentes, uma floresta de isolamento e uma rede neural, sobre exatamente o mesmo pipeline de dados.

Responsabilidades dos módulos

Módulo	Responsabilidade	Entrada → Saída
<code>data/loader.py</code>	ler os ZIPs e normalizar cabeçalhos	pasta → 4 tabelas NumPy
<code>preprocessing/transform.py</code>	converter valores e datas, padronizar, dividir	texto → números
<code>features/feature.py</code>	juntar as 3 tabelas e agregar por licitação	3 tabelas → 8.064×18
<code>models/isolation_forest.py</code>	modelo principal (implementação própria)	matriz → score 0–1
<code>models/autoencoder.py</code>	modelo de comparação (nn.Module)	matriz → erro de reconstrução
<code>training/</code>	treinar e salvar artefatos	matriz → modelo + scaler
<code>evaluation/evaluate.py</code>	régua de risco, precisão@k, ranking	scores → relatório
<code>inference/inference.py</code>	pontuar novos lotes	features → anomalias

Como o main.py integra: é a única peça que conhece todas as outras — chama `load_tables` → `build_feature_matrix` → `standardize` → `split` → `treinar` → `salvar` → `relatório`.

Código, NumPy e tipagem

Interfaces explícitas e responsabilidade única — três exemplos do projeto

```
def parse_valor(values: np.ndarray) -> np.ndarray:
```

Converte valores monetários brasileiros ('1.234,56') para float64.

► Tipagem: entrada e saída explícitas, quem chama sabe o que recebe.

```
def standardize(data: np.ndarray) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
```

Padroniza (z-score) e devolve TAMBÉM mean e std.

► O tipo denuncia que devolve 3 coisas: a inferência precisa guardar mean/std.

```
def _unique_por_grupo(grupo_id: np.ndarray, valores: np.ndarray,  
                      n_grupos: int) -> np.ndarray:
```

Conta valores distintos por grupo.

► NumPy: um 'groupby' vetorizado com np.unique e np.bincount, sem pandas.

Implementação do modelo em PyTorch

Isolation Forest — PRINCIPAL

implementação própria do grupo

```
def compute_anomaly_score(
    self, X, device: str = "cpu"):
    X_t = torch.tensor(X, dtype=torch.float32,
                       device=device)

    ...
    return 2.0 ** (-(avg_paths /
                    self.c_subsample))
```

- Escrito do zero: árvores de isolamento, c_factor , comprimento de caminho
- Não usa scikit-learn, o score é vetorizado em PyTorch, com device CPU/GPU
- Salvamento: pickle (a floresta é estrutura de árvores Python/NumPy)

Autoencoder — comparação

nn.Module (PyTorch)

```
class Autoencoder(nn.Module):
    def __init__(self, input_dim: int,
                 hidden1=32, hidden2=16,
                 bottleneck=8):
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden1),
            nn.ReLU(), ...)
```

Arquitetura	18 → 32 → 16 → 8 → 16 → 32 → 18
Perda / otimizador	nn.MSELoss() · Adam, lr = 1e-3
Épocas / batch	60 · 64
CPU / GPU	experimentos rodaram em CUDA
Salvar / carregar	torch.save(state_dict) / load_state_dict

Configuração experimental

Comparação entre o modelo principal e a alternativa avaliada

Divisão

80% treino / 20% teste · semente 42

Modelos comparados

Isolation Forest (principal) × Autoencoder

Hiperparâmetros — IF

100 árvores · max_samples = 256

Hiperparâmetros — AE

60 épocas · batch 64 · lr 1e-3

Métrica

precisão@k contra a régua de risco (0–4 bandeiras)

Limiar

percentil 95 do score de anomalia

Experimentos anteriores

24 configurações de autoencoder (notebook, entrega 4)

Base

8.064 licitações × 18 features

O que queríamos descobrir

Qual detector prioriza melhor as licitações que carregam as bandeiras vermelhas clássicas da literatura de auditoria?

Ressalva: os 24 experimentos do notebook rodaram sobre 1 feature; a comparação IF × AE roda sobre as 18.

Testes automatizados e evidências

```
def test_treino_e_reprodutivel(self):  
    """Mesma semente => mesmo resultado.  
    É o RNF de reprodutibilidade na prática."""  
    _m1, h1 = self._treinar()  
    _m2, h2 = self._treinar()  
    self.assertEqual(h1.train_loss, h2.train_loss)  
  
def test_modelo_salvo_pode_ser_recarregado(self):  
    """Não basta o arquivo existir: o modelo  
    recarregado tem que reconstruir igual."""  
    self.assertTrue(  
        torch.allclose(saida_original,  
                       saida_carregada))
```

Evidência da execução

Ran 26 tests in 0.862s

OK

Cobertura da suíte

test_data — loader e ZIPs

test_preprocessing — datas e split

test_model — formato e save/load

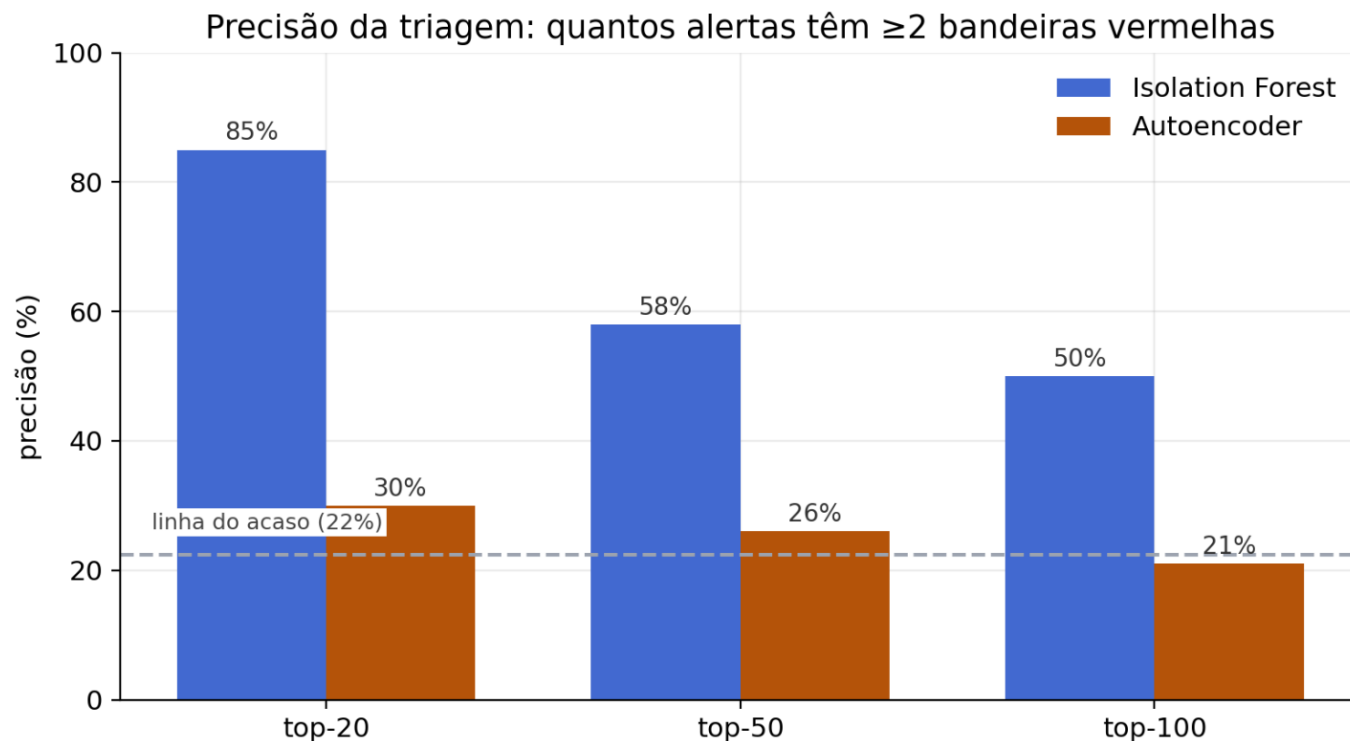
test_training — lotes, perda, artefatos

Por que estes dois testes

O primeiro transforma um requisito não funcional (reprodutibilidade) em verificação automática. O segundo não se contenta com o arquivo existir, exige que o modelo recarregado produza exatamente a mesma saída, que é o que a inferência depende.

Resultados, conclusão e limitações

O Isolation Forest quase quadruplica a taxa do acaso no top-20



Alerta nº 1 do ranking

Inexigibilidade · DF
R\$ 389.475.750,00
1 único participante
Risco 4 de 4 na régua

Requisitos atendidos

RF-01 e RF-03 implementados.
RF-02, RF-04, RF-05 e RF-06 no roadmap, o projeto é uma maquete funcional, como previsto.

A escolha feita no GR4ML por explicabilidade também se mostrou a de melhor desempenho — e o modelo é 10× mais rápido (0,27 s contra 2,82 s).

Limitações

A régua usa as mesmas features do modelo · sem retorno dos auditores · notebooks usaram 1 feature

Próximos passos

Integrar SHAP · cruzar CEIS/CNEP · modelo em nível de item para sobrepreço